Projet 1 : un clone de egrep

DAAR 2026-2027, Sorbonne Universite, M2 Informatique (RES-ALT-SOFT)

Recherche de motif par expressions regulieres : automates finis, KMP et Boyer-Moore

Auteur(s) : NOM1, NOM2

Rendu du 31 octobre 2026

1. Introduction

Ce projet realise un clone de la commande **egrep** : etant donne une expression reguliere et un fichier texte, le programme affiche les lignes qui contiennent une occurrence du motif. Nous traitons le sous-ensemble de la norme ERE impose par le sujet (parentheses, alternative, concatenation, etoile, point, lettre ASCII) et nous ajoutons **+**, **?** et l'echappement par ****. Le programme est ecrit en Java. Trois familles d'algorithmes sont implementees : la chaine complete de la seance 1 (arbre de syntaxe, automate non deterministe, automate deterministe, automate minimal), les algorithmes de recherche de facteur **Knuth-Morris-Pratt** et **Boyer-Moore** lorsque le motif est une simple concatenation de lettres, et une simulation directe de l'automate non deterministe, utilisee comme point de comparaison. Le resultat est valide contre egrep sur plusieurs textes et motifs, puis les performances sont mesurees.

2. Definition du probleme et structures de donnees

Probleme. Soit un motif R (une RegEx) et un fichier de l lignes. On veut les lignes u telles qu'un facteur de u appartient au langage L(R). Comme egrep, on teste chaque ligne independamment, et une ligne est retenue des qu'une occurrence existe, quelle que soit sa position.

Structures de donnees. L'arbre de syntaxe est un arbre binaire de noeuds (type, lettre, fils gauche, fils droit). L'automate non deterministe stocke pour chaque etat une liste d'arcs (symbole, cible), avec un symbole special pour l'epsilon et un autre pour le point. L'automate deterministe est une table de transitions *next[etat][octet]* sur l'alphabet de 256 octets, avec un tableau de booleens pour les etats acceptants. Les fichiers sont lus en octets (ISO-8859-1) et parcourus sans conversion en chaines, ce qui evite de dependre de l'encodage et correspond au comportement de egrep en locale C. Les fins de ligne sont uniquement les octets \n, un \r eventuel reste dans la ligne comme dans egrep.

3. Algorithmes

3.1 Chaine automate (seance 1)

(1) Parsing. Une descente recursive suit la grammaire : alternative (priorite la plus faible), concatenation, repetition (*, +, ?), atome (lettre, point, expression entre parentheses). L'arbre de *S(a|g|r)+on* est la concatenation de S, du plus de l'alternative a|g|r, de o et de n.

(2) Aho-Ullman. Chaque noeud de l'arbre est traduit par un fragment d'automate a un etat initial et un etat final, relies par des epsilon-transitions : lettre (1 arc), concatenation (epsilon entre les deux fragments), alternative (deux epsilon en entree et en sortie), etoile (retour arriere et saut). Le nombre d'etats est lineaire en la taille du motif (au plus deux par noeud de l'arbre).

Recherche d'un facteur. Plutot que de tester tous les suffixes d'une ligne, on construit l'automate de $\Sigma^*.R$: un nouvel etat initial boucle sur tous les symboles et possede un epsilon vers l'initial de R. La ligne est acceptee si cet automate atteint un etat acceptant a un instant quelconque. Un seul parcours lineaire suffit.

(3) Sous-ensembles. Chaque etat du deterministe est un ensemble d'etats du NFA, clos par epsilon. On calcule pour chacun des 256 octets l'ensemble atteint. Nous rendons les etats acceptants *absorbants* : des qu'une occurrence est trouvee, la ligne est retenue, la suite est sans importance. Cette simplification reduit le nombre d'etats et fait que tous les acceptants se fusionnent a l'etape suivante. Dans le pire cas le deterministe a 2^n etats, mais les motifs usuels restent petits (tableau 1).

(4) Minimisation. Nous utilisons l'algorithme de Moore (raffinement de partition) : partition initiale acceptants / non acceptants, puis separation des etats dont les successeurs sont dans des classes differentes, jusqu'a stabilite. Sa complexite est $O(n^2|\Sigma|)$ dans le pire cas. L'algorithme de Hopcroft, en $O(n|\Sigma| \log n)$, serait preferable pour de tres gros automates, mais ici n reste petit et Moore est plus simple a justifier.

(5) Recherche. Pour chaque ligne, on suit les transitions octet par octet depuis l'etat initial et on s'arrete des qu'un etat acceptant est atteint : O(|ligne|) par ligne, donc O(taille du fichier) au total, independamment de la complexite du motif.

3.2 Knuth-Morris-Pratt

Lorsque le motif est un mot m de longueur k, KMP evite de revenir en arriere dans le texte. Le precalcul construit la liste *Retenue* : retenue[i] est la longueur du plus long bord propre du prefixe de longueur i (le plus long mot a la fois prefixe et suffixe strict). En cas d'echec apres k' lettres correspondantes, on repart a la position retenue[k'] sans reculer dans le texte. Le precalcul coute O(k) et la recherche O(n) comparaisons au plus, dans tous les cas, ce qui garantit le pire cas lineaire.

3.3 Boyer-Moore

Boyer-Moore compare le motif a la fenetre courante de **droite a gauche**. A l'echec, il decale le motif du maximum de deux heuristiques. *Mauvais caractere* : on aligne la derniere occurrence dans le motif du caractere fautif du texte (tableau de 256 entiers). *Bon suffixe* : on aligne le suffixe deja reconnu avec une autre occurrence de lui-meme dans le motif, ou avec un de ses bords (tableau calcule en O(k)). Le gain vient de ce que, sur un texte naturel, la plupart des fenetres echouent des la premiere comparaison avec un grand decalage (jusqu'a k) : le nombre moyen de comparaisons par octet est nettement inferieur a 1, et il diminue quand le motif s'allonge. Dans le pire cas la version simple coute O(nk) comparaisons ; la regle de Galil permet de le ramener a O(n).

3.4 Remarques et critique

KMP et Boyer-Moore ne couvrent pas le cas general : ils exigent un motif sans operateur. Le programme detecte ce cas dans l'arbre de syntaxe. **La simulation du NFA** est en O(|ligne| × nombre d'etats), contre O(|ligne|) pour le deterministe : la determinisation deplace le cout dans un precalcul qui ne depend pas du texte. **Limite de notre implementation** : les tables de 256 colonnes sont gourmandes en memoire si le deterministe grossit, une compression de l'alphabet (classes d'octets equivalents) serait une amelioration directe. Enfin egrep fait mieux que tous nos algorithmes car il combine memchr, Boyer-Moore-Horspool sur les facteurs obligatoires du motif et automates paresseux, le tout en C et en lisant par gros blocs.

4. Tests

Validite. Le script *tests/run_tests.sh* compare octet pour octet la sortie du clone et celle de **egrep** (locale C) sur 14 expressions (alternatives, etoiles imbriquees, point, motif vide, mots simples) et sur plusieurs textes, pour les algorithmes dfa, nfa, kmp et bm. Tous les tests passent. Les motifs litteraux incluent des cas a forts bords (*aaab*, *ababab*, *aabaab*) pour exercer les tables de KMP et du bon suffixe, sur un texte binaire a deux lettres.

Testbed de performance. Les mesures du tableau et de la figure ci-dessous ont ete faites sur un texte synthetique de 11 Mo (environ 300 000 lignes, mots tires au hasard dans un vocabulaire d'une trentaine de mots), faute d'avoir pu utiliser les livres Gutenberg dans l'environnement de

developpement. Les motifs suivent la suggestion du TME2 : Chihuahua, Pizzi, Pepperoni, Pizza, Poppers, plus trois motifs avec operateurs. Chaque mesure est repetee 5 fois (moyenne et ecart type). Pour le clone on mesure le temps de recherche interne, hors demarrage de la JVM, hors lecture du fichier et hors precalcul ; pour egrep on ne peut que mesurer le temps de la commande complete avec l'option -c. La comparaison avec egrep est donc a l'avantage du clone et ne doit se lire que comme un ordre de grandeur.

5. Resultats

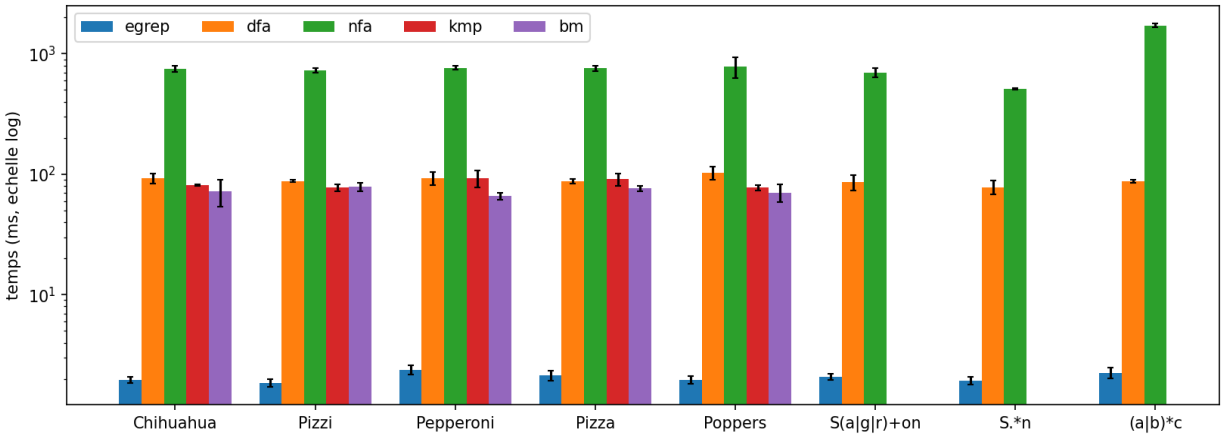


Figure 1 : temps de recherche moyen (ms, echelle logarithmique, barres d'erreur = ecart type), 5 executions.

Motif	NFA	DFA	DFA min.
S(a|g|r)+on	25	8	6
Chihuahua	35	11	11
(a|b)*c	13	5	3
S.*n	13	6	4

Tableau 1 : nombre d'etats des automates (motif non ancre Σ*.R).

6. Discussion

Automate contre simulation. Le deterministe est entre 6 et 20 fois plus rapide que la simulation du NFA (par exemple 88 ms contre 758 ms pour *Pizza*, 88 ms contre 1715 ms pour *(a|b)*c*). Plus le NFA a d'etats actifs simultanes (cas de *(a|b)*c*), plus l'ecart augmente. Le temps du DFA ne depend presque pas du motif : c'est la propriete attendue, un acces par octet.

KMP et Boyer-Moore. Sur les motifs litteraux, KMP est proche du DFA (le DFA d'un mot est precisement l'automate de KMP complet), alors que Boyer-Moore est un peu plus rapide sur les motifs longs (*Pepperoni* : 66 ms contre 92 ms pour KMP), et equivalent sur les motifs courts comme *Pizzi*, ou l'ecart est dans l'ecart type. Cela va dans le sens de la theorie : Boyer-Moore decale d'autant plus que le motif est long, alors que KMP lit chaque octet. L'ecart reste modeste car notre mesure inclut le decoupage en lignes, commun aux algorithmes, qui represente une part importante du temps ; l'effet serait plus net sur un texte plus gros ou avec un decoupage en lignes evite.

egrep. egrep reste tres en avance, d'un facteur 40 environ. Il faut y voir le cout d'une implementation Java (verification des bornes, pas de vectorisation) et surtout les optimisations de GNU grep : recherche de facteurs obligatoires, Boyer-Moore-Horspool vectorise, absence de

decoupage ligne par ligne tant qu'aucune occurrence n'est trouvee. Notre decoupage ligne par ligne a un cout non negligeable que l'on n'a pas cherche a eliminer.

Limites. Les resultats dependent du texte et de la machine. Les nombres precis sont a lire comme des tendances. Les textes de Gutenberg ont des distributions de lettres differentes de celles de notre texte, ce qui joue surtout sur Boyer-Moore.

7. Conclusion et perspectives

Nous avons implemente une chaine complete de la RegEx vers l'automate minimal, validee contre egrep, et deux algorithmes de recherche de facteur. Le deterministe offre un temps lineaire independant du motif au prix d'un precalcul, Boyer-Moore est legerement plus rapide que KMP sur les mots longs. Perspectives : un index de mots (forward index, P-trie ou filtre de Bloom) calcule une fois et mis en cache pour accelerer les recherches repetees sur un meme texte ; un automate paresseux (determinisation a la demande) pour eviter l'explosion d'etats ; la minimisation de Hopcroft ; l'extraction d'un facteur obligatoire du motif pour utiliser Boyer-Moore comme pre-filtre avant l'automate, comme le fait egrep ; enfin le support des classes de caracteres et des ancres, hors du cadre du sujet.